



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования

«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
Департамент программной инженерии и искусственного интеллекта

ОТЧЕТ
о выполнении индивидуального задания по теме «Разработка
технической документации»

в рамках освоения дисциплины
«Основы алгоритмизации и программирования»
направления «Программная инженерия»

Отчет защищен с оценкой

_____ О.В. Дудко

(подпись)

«_____» _____ 2026 г.

Выполнил студент

группы Б9125-09.03.04пи

_____ А.Е. Мохнова

(подпись)

Руководитель

Доцент ДПИиИИ, Кандидат

физико-математических наук О.В.

Дудко

Владивосток

2026

Аннотация

Алгоритм решения задачи реализован в виде программного кода, написанного на алгоритмическом языке C++.

Общее назначение программы: обработка данных о студентах вуза, нахождение и вывод в файл информации о (1) номере курса, на котором учится наибольший процент парней, (2) трех самых распространенных мужских имени в списке, (3) фамилии и имени самой младшей девушки и самого младшего парня в списке. Также вывод сообщения об ошибке в случае ввода некорректных входных данных.

Проект программы разработан с использованием принципов нисходящего проектирования; алгоритм программы основан на принципе структурного программирования.

Оглавление

Неформальная постановка задачи.....	4
Формальная постановка задачи.....	4
Алгоритм решения задачи	7
Спецификация данных	13
Спецификация функций.....	13
Проектирование программы.....	15
Описание данных программы	16
Алгоритм программы на языке PDL.....	20
Тесты.....	26
Тестирование программы	38
Список использованных источников.....	40

Неформальная постановка задачи

Дана информация о каждом из 2000 студентов вуза: <фамилия>, <имя>, <отчество>, <пол>, <возраст>, <курс>. В фамилии, имени и отчестве не более 12 букв, пол указывается буквами М или Ж, возраст – целое число от 16 до 35, курс – целое число от 1 до 6. Необходимо найти:

1 Номер курса, на котором учится наибольший процент парней. Если в списке нет парней, вместо процента выводится сообщение об этом.

2 Три самых распространенных мужских имени в списке (по алфавиту). Если различных мужских имен в файле меньше трех, то выводится 2 имени, 1 или ни одного (в этом случае вместо имен выводится сообщение об отсутствии мужских имен в списке).

3 Фамилию и имя самой младшей девушки и самого младшего парня в списке. В случае отсутствия девушек либо парней в списке, вместо фамилии и имени выводится сообщение об этом.

Если входной файл пуст, то должно быть выведено сообщение об этом. При обнаружении ошибок формата входных данных (недостаточно полей, пустое поле, неверный формат поля, значение не в допустимом диапазоне) программа должна вывести сообщение об ошибке и завершить работу. Если обнаружена пустая строка, то она пропускается, программа продолжает работу.

Если невозможно открыть входной файл, выводится сообщение об ошибке на экран и в выходной файл. Если невозможно открыть выходной файл, выводится сообщение об ошибке только на экран.

Формальная постановка задачи

Введем обозначения:

$string = \{e_i : e_i \in \{\langle a \rangle .. \langle z \rangle, \langle A \rangle .. \langle Z \rangle, \langle - \rangle, \langle \rangle\}\}$

$nameType = \{e_j : e_j \in \{\langle a \rangle .. \langle z \rangle, \langle A \rangle .. \langle Z \rangle\}, 1 \leq j \leq 12\}$

Тогда

Input:

outFileName $\in$ string – Имя выходного файла

$A = \{a_i : a_i = \{sn_i, n_i, fn_i, g_i, age_i, y_i\}, 1 \leq i \leq 2000\}$ – множество записей о студентах, где

$sn_i \in nameType$ – Фамилия

$n_i \in nameType$ – Имя

$fn_i \in nameType$ – Отчество

$g_i \in \{\langle M \rangle, \langle F \rangle\}$ – Пол

$age_i \in \mathbb{N}, 16 \leq age_i \leq 35$ – Возраст

$y_i \in \mathbb{N}, 1 \leq y_i \leq 6$ – Курс

Output:

outMessage $\in \{ (maxpr, mostPopularNames, mminan, fminan), (lineid, segment), r \}$, где

1 $maxpr \in \{ \{ \mathbb{N}, 1 \leq maxpr \leq 6 \}, yr \in string \}$ – Номер курса с наибольшим процентом парней, сообщение об отсутствии парней.

2 $mostPopularNames \in \{ \{ mpn_k \in \{n_i\}, 1 \leq k \leq 3 \}, mpnr \in string \}$ – Самые распространенные мужские имена, сообщение об отсутствии мужских имен.

3 $mminan \in \{ (msn, mn) : msn \in \{n_i\}, mminr \in string \}$ – Фамилия и имя самого младшего парня, сообщение об отсутствии парней.

$fminan \in \{ (fsn, fn) : fsn \in \{n_i\}, fminr \in string \}$ – Фамилия и имя самой младшей девушки, сообщение об отсутствии девушек.

$lineid \in \mathbb{N}, 1 \leq lineid \leq 2000$ – номер строки с ошибкой

$segment \in \{ \text{“surname”, “name”, “father name”, “gender”, “age”, “year”} \}$ – имя сегмента с ошибкой

$r \in string$ – Сообщение об ошибке.

Связи:

1) Пусть $P = \{0, 0, 0, 0, 0, 0\}$

Для $\forall ru_j \in P$

$M = 0; C = 0$

Для $\forall a_i$

Если $a_i.y_i == j$, то

Если $a_i.g_i == 'M'$, то $M = M + 1$

$C = C + 1$

$ru_j = M / C$

Если $\max(P) > 0$, то $\max ru = j : ru_j = \max(P)$

Иначе $\max ru = ur$

2) $MNC = \{ (mnc_j, mn_j) : mnc_j \in \mathbb{N}, mn_j \in \{n_i\} \}$

Для $\forall a_i$

Если $a_i.g_i == 'M'$, то

Если $(0, a_i.n_i) \notin MNC$, то $MNC = MNC \cup (0, a_i.n_i)$

$(mnc_i, mn_i) = (mnc_i + 1, mn_i)$

$SN = \text{Сортировать}(MNC)$ по убыванию mnc_j , при равенстве - по возрастанию mn_j

Если $|MNC| > 0$, то

$mostPopularNames = \{ mpn_k \in SN, 1 \leq k \leq |MNC| \leq 3 \}$

Иначе $mostPopularNames = mpnr$

3) $minMA = +\infty$

$minFA = +\infty$

$mminan = mminr$

$fminan = fminr$

Для $\forall a_i$

Если $a_i.g_i == 'M'$ и $a_i.age_i < minMA$, то

$minMA = a_i.age_i$

$mminan = (a_i.sn_i, a_i.n_i)$

Если $a_i.g_i == 'F'$ и $a_i.age_i < minFA$, то

$minFA = a_i.age_i$

$f_{\min} = (a_i.s_n_i, a_i.n_i)$

Если $|A| > 0$, то $outMessage = (maxpy, mostPopularNames, m_{\min}, f_{\min})$

Иначе $outMessage = r$

Алгоритм решения задачи

0. Начало

1. Ввод имени выходного файла в $outFileName$

2. Открытие файла «in1.txt» для чтения в поток in

3. Если НЕ(in открыт), то

3.1. $outMessage = \text{«Input file open error»}$

3.2. Открытие выходного файла $\langle outFileName \rangle.txt$ в поток out

3.3. Если out открыт, то

3.3.1. Запись $outMessage$ в out

3.3.2. Вывод $outMessage$ на экран

3.4. Иначе

3.4.1. Вывод «Output file open error» на экран

3.5. Переход к шагу 13

4. Инициализировать список $students$

5. Инициализировать пустые строки $outMessage$, $line$

6. $i = 1$

7. Пока (есть строки в in) И ($i \leq 2000$)

7.1. Записать строку из in в $line$

7.2. Если $line$ пустая, то

7.2.1. Перейти к следующей итерации

7.3. Инициализировать переменные для полей: пустая строка $surName$,

пустая строка $name$, пустая строка $fatherName$, символ $gender$, число

age , число $year$

7.4. $pid = 0$

7.5. Записать в id индекс первого вхождения символа «;» в $line$

- 7.6. Записать в surName результат readSegment(line, pid, id)
- 7.7. Если НЕ ((длина surName $\leq$ 12) И (длина surName $>$ 0)), то
 - 7.7.1. Вызвать formatError(i, «surname», outFileName)
 - 7.7.2. Переход к шагу 13
- 7.8. Записать в name результат readSegment(line, pid, id)
- 7.9. Если НЕ ((длина name $\leq$ 12) И (длина name $>$ 0)), то
 - 7.9.1. Вызвать formatError(i, «name», outFileName)
 - 7.9.2. Переход к шагу 13
- 7.10. Записать в fatherName результат readSegment(line, pid, id)
- 7.11. Если НЕ ((длина fatherName $\leq$ 12) И (длина fatherName $>$ 0)), то
 - 7.11.1. Вызвать formatError(i, «father name», outFileName)
 - 7.11.2. Переход к шагу 13
- 7.12. genderAllowed = {«M», «F»}
- 7.13. Записать в sgender результат readSegment(line, pid, id)
- 7.14. Если длина sgender == 1, то
 - 7.14.1. Если genderAllowed содержит sgender[0], то
 - 7.14.1.1. Записать в gender значение sgender как символ
 - 7.14.2. Иначе
 - 7.14.2.1. Вызвать formatError(i, «gender», outFileName)
 - 7.14.2.2. Переход к шагу 13
- 7.15. Иначе
 - 7.15.1. Вызвать formatError(i, «gender», outFileName)
 - 7.15.2. Переход к шагу 13
- 7.16. Записать в sage результат readSegment(line, pid, id)
- 7.17. Если (длина sage == 2) И isNumeric(sage), то
 - 7.17.1. Записать в age числовое значение sage
 - 7.17.2. Если НЕ ((age $\geq$ 16) И (age $\leq$ 35)), то
 - 7.17.2.1. Вызвать formatError(i, «age», outFileName)
 - 7.17.2.2. Переход к шагу 13
- 7.18. Иначе

- 7.18.1. Вызвать `formatError(i, «age», outFileName)`
- 7.18.2. Переход к шагу 13
- 7.19. Записать в `syear` результат `readSegment(line, pid, id)`
- 7.20. Если (длина `syear == 1`) И `isNumeric(syear)`, то
 - 7.20.1. Записать в `year` числовое значение `syear`
 - 7.20.2. Если НЕ $((year \geq 1) \text{ И } (year \leq 6))$, то
 - 7.20.2.1. Вызвать `formatError(i, «year», outFileName)`
 - 7.20.2.2. Переход к шагу 13
- 7.21. Иначе
 - 7.21.1. Вызвать `formatError(i, «year», outFileName)`
 - 7.21.2. Переход к шагу 13
- 7.22. Инициализировать объект `Student student` с полями со значениями `surName, name, fatherName, gender, age, year`
- 7.23. Добавить `student` в `students`
- 7.24. $i = i + 1$
- 7.25. Перейти к следующей строке в `in`
- 8. Если НЕ(`students` пуст), то
 - 8.1. $maxp = 0, maxpy = 1$
 - 8.2. Для i от 1 до 6
 - 8.2.1. $c = 0, mc = 0$
 - 8.2.2. Для каждого s в `students`
 - 8.2.2.1. Если $s.year == i$, то
 - 8.2.2.1.1. Если $s.gender == \text{«М»}$, то
 - 8.2.2.1.1.1. $mc = mc + 1$
 - 8.2.2.1.2. $c = c + 1$
 - 8.2.3. $p = mc / c$
 - 8.2.4. Если $maxp < p$, то
 - 8.2.4.1. $maxp = p$
 - 8.2.4.2. $maxpy = i$
- 8.3. Если $maxp == 0$, то

- 8.3.1. Добавить к outMessage «Max men: <maxpy> year \n»
- 8.4. Иначе
 - 8.4.1. Добавить к outMessage «Max men: no men in list \n»
- 8.5. Инициализировать пустые множества names, mostPopularNames
- 8.6. Для каждого s в students
 - 8.6.1. Если s.gender == «М», то
 - 8.6.1.1. Добавить s.name в names
- 8.7. Для i от 0 до 2
 - 8.7.1. maxc = -1
 - 8.7.2. Инициализировать пустую строку maxcn
 - 8.7.3. Для каждого name в names
 - 8.7.3.1. Если mostPopularNames не содержит name, то
 - 8.7.3.1.1. c = 0
 - 8.7.3.1.2. Для каждого s в students
 - 8.7.3.1.2.1. Если s.name == name, то
 - 8.7.3.1.2.1.1. c = c + 1
 - 8.7.3.1.3. Если c > maxc, то
 - 8.7.3.1.3.1. maxc = c
 - 8.7.3.1.3.2. maxcn = name
 - 8.7.4. Добавить maxcn в mostPopularNames
 - 8.8. Добавить к outMessage «Most popular male names: »
 - 8.9. Если НЕ(mostPopularNames пуст), то
 - 8.9.1. Для каждого n в mostPopularNames
 - 8.9.1.1. Добавить к outMessage «<n> »
 - 8.10. Иначе
 - 8.10.1. Добавить к outMessage «no male names»
 - 8.11. Добавить к outMessage знак перехода на новую строку
 - 8.12. mmina = 100, fmina = 100
 - 8.13. mminan = «no males»
 - 8.14. fminan = «no females»

- 8.15. Для каждого s в students
 - 8.15.1. Если s.gender == «М», то
 - 8.15.1.1. Если s.age < mmina, то
 - 8.15.1.1.1. mmina = s.age
 - 8.15.1.1.2. Записать в mminan «<s.name> <s.surName>»
 - 8.15.2. Иначе
 - 8.15.2.1. Если s.age < fmina, то
 - 8.15.2.1.1. fmina = s.age
 - 8.15.2.1.2. Записать в fminan «<s.name> <s.surName>»
- 8.16. Добавить к outMessage «Youngest male: <mminan> \n»
- 8.17. Добавить к outMessage «Youngest male: <fminan> \n»
9. Иначе
 - 9.1. Записать в outMessage «The input file is empty»
10. Открытие выходного файла <outFileName>.txt в поток out
11. Если out открыт, то
 - 11.1. Запись outMessage в out
 - 11.2. Вывод outMessage на экран
12. Иначе
 - 12.1. Вывод «Output file open error» на экран
13. Конец

Функция isNumeric(s)

1. Начало
2. Инициализировать множество numbers = {«0», «1», «2», «3», «4», «5», «6», «7», «8», «9»}
3. Для каждого с в s
 - 3.1. Если НЕ numbers содержит с, то
 - 3.1.1. Вернуть false
4. Вернуть true

Функция readSegment(line, pid, id)

1. Начало
2. res = 0
3. Если символ “;” был найден, то
 - 3.1. Если pid != 0, то
 - 3.1.1. pid = pid + 1
 - 3.2. Записать в res подстроку line из (id - pid) символов с индекса pid
4. pid = id
5. Записать в id индекс первого вхождения символа «;», начиная с индекса pid + 1
6. Вернуть res

Процедура formatError(lineid, segment, outFileName)

1. Начало
2. Инициализировать пустую строку outMessage
3. Записать в outMessage «line <lineid>: format error (<segment>)»
4. Открытие выходного файла <outFileName>.txt в поток out
5. Если out открыт, то
 - 5.1. Запись outMessage в out
 - 5.2. Вывод outMessage на экран
6. Иначе
 - 6.1. Вывод «Output file open error» на экран
7. Конец

Спецификация данных

Спецификация ввода

Ввод имени выходного файла выполняется с клавиатуры:

<outFileName>

Ввод данных о студентах выполняется из файла «in1.txt» построчно.

Формат строки файла:

<surName>;<name>;<fatherName>;<gender>;<age>;<year>;

Спецификация вывода

Вывод сообщений 1-4 выполняется на экран и в файл <outFileName>.txt. Вывод сообщения 5 – только на экран.

Сообщение 1:

«Max men: » + <maxpy> + «\n Most popular male names: » + <mostPopularNames> + «\n» + «Youngest male: » + <mminan> + «\n» + «Youngest female: » + <fminan> + «\n»

Сообщение 2:

«The input file is empty»

Сообщение 3:

«line » + <lineid> + «: format error (» + <segment> + «)»

Сообщение 4:

«Input file open error»

Сообщение 5:

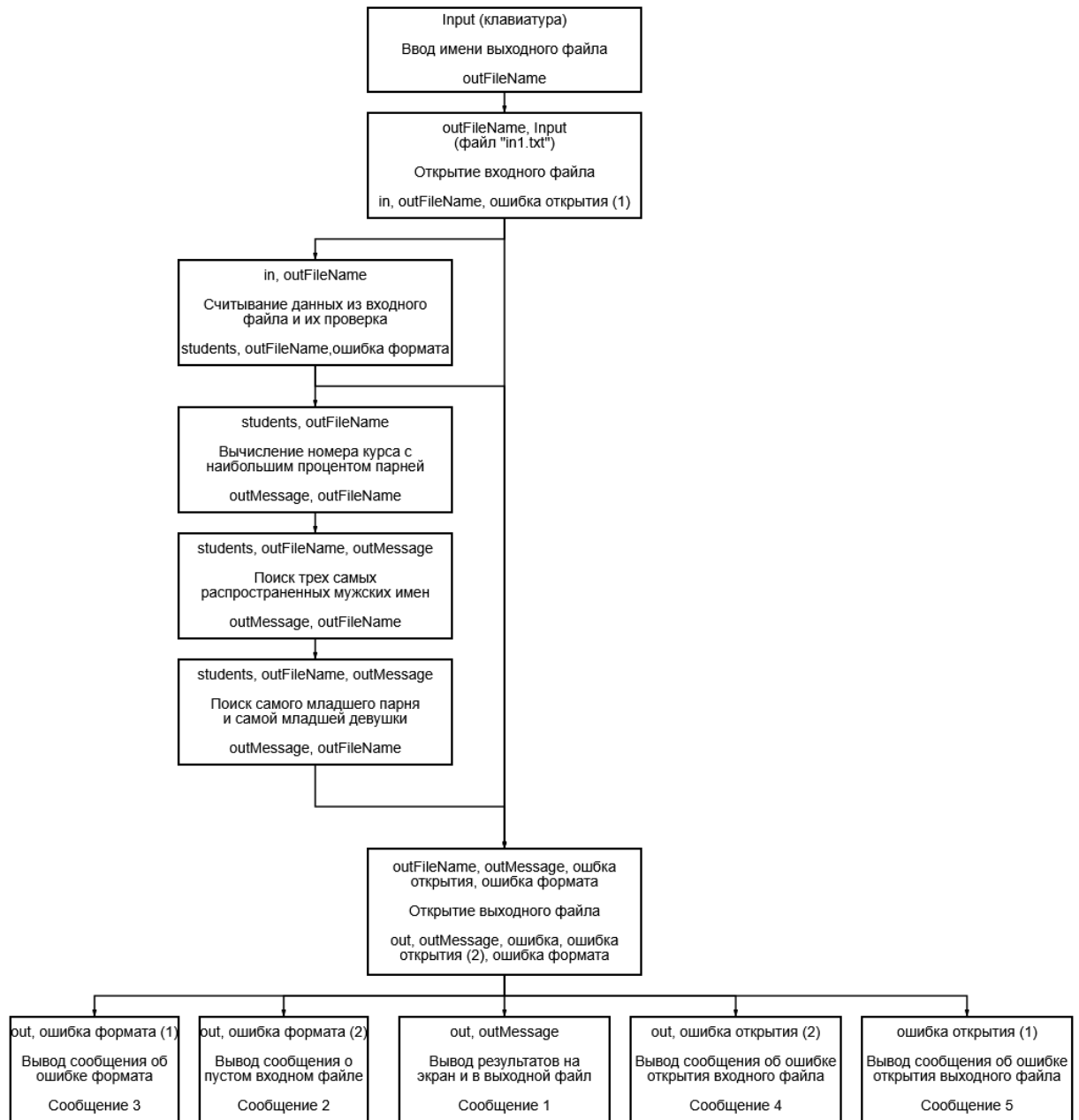
«Output file open error»

Спецификация функций

- 1 Ввод имени выходного файла.
- 2 Открытие входного файла.
- 3 Вывод сообщения об ошибке открытия входного файла.
- 4 Считывание данных из входного файла и их проверка.

- 5 Вывод сообщения об ошибке формата.
- 6 Вывод сообщения о пустом входном файле.
- 7 Вычисление номера курса с наибольшим процентом парней.
- 8 Поиск трех самых распространенных мужских имен.
- 9 Поиск самого младшего парня и самой младшей девушки.
- 10 Открытие выходного файла.
- 11 Вывод сообщения об ошибке открытия выходного файла.
- 12 Вывод результатов на экран и в выходной файл.

Проектирование программы



Описание данных программы

Таблица 1 – Переменные

Идентификатор	Назначение	Тип
Student (struct)		
surName	Фамилия студента	string
name	Имя студента	string
fatherName	Отчество студента	string
gender	Пол студента	char
age	Возраст студента	int
year	Курс студента	int
Глобальный контекст		
outFileName	Имя выходного файла	string
in	Входной файловый поток	ifstream
students	Вектор для хранения данных о студентах	vector<Student>
outMessage	Сообщение для вывода результатов	string
line	Текущая строка из входного файла	string
i	Счетчик строк/номер текущей строки	int
surName	Фамилия текущего студента	string
name	Имя текущего студента	string
fatherName	Отчество текущего студента	string
gender	Пол текущего студента	char
age	Возраст текущего студента	int
year	Курс текущего студента	int
pid	Позиция предыдущего	size_t

	разделителя «;»	
id	Позиция текущего разделителя «;»	size_t
genderAllowed	Множество допустимых символов пола	set<char>
sgender	Строковое представление пола	string
sage	Строковое представление возраста	string
syear	Строковое представление курса	string
student	Объект студента с прочитанными данными	Student
maxp	Максимальный процент парней на курсе	float
maxpy	Номер курса с максимальным процентом парней	int
c	Счетчик студентов на текущем курсе	float
mc	Счетчик парней на текущем курсе	float
p	Процент парней на текущем курсе	float
names	Множество уникальных мужских имен	set<string>
mostPopularNames	Множество трёх самых популярных мужских имен	set<string>

maxc	Максимальное количество вхождений имени	int
maxcn	Имя с максимальным количеством вхождений	string
n	Текущее имя из множества популярных имен	string
mmina	Минимальный возраст среди парней	int
fmina	Минимальный возраст среди девушек	int
mminan	Имя и фамилия самого младшего парня	string
fminan	Имя и фамилия самой младшей девушки	string
out	Выходной файловый поток	ofstream
isNumeric	Проверка, является ли строка числом	bool
formatError	Вывод сообщения об ошибке формата в строке	void
readSegment	Чтение сегмента строки и переход к следующему	string
Локальный контекст isNumeric		
s	Проверяемая строка	string
numbers	Множество цифровых символов	set<char>
c	Текущий символ строки	char
Локальный контекст readSegment		
line	Обрабатываемая строка	string
pid	Ссылка на позицию	size_t&

	предыдущего разделителя	
id	Ссылка на позицию текущего разделителя	size_t&
res	Результат (извлеченный сегмент строки)	string
Локальный контекст formatError		
lineid	Номер строки с ошибкой формата	int
segment	Название поля с ошибкой	string
outFileName	Имя выходного файла	string
outMessage	Сообщение об ошибке	string
out	Выходной файловый поток	ofstream

Алгоритм программы на языке PDL

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <set>
using namespace std;

struct Student
{
    string surName{};
    string name{};
    string fatherName{};

    char gender{};
    int age{};
    int year{};

    Student(string surName, string name, string fatherName,
            char gender, int age, int year)
        : surName{ surName }, name{ name }, fatherName{ fatherName },
        gender{ gender }, age{ age }, year{ year }
    {
    };
};

bool isNumeric(string s);
string readSegment(string line, size_t& pid, size_t& id);
void formatError(int lineid, string segment, string outFileName);

int main()
{
    string outFileName;

    cout << "Enter output file name: ";
    getline(cin, outFileName);

    ifstream in;
    in.open("in1.txt");

    if (!in)
    {
        string outMessage{ "Input file open error" };
        ofstream out;
        out.open(outFileName + ".txt");

        if (out)
        {
            out << outMessage;
            cout << outMessage;
        }
        else cout << "Output file open error" << endl;

        return 0;
    }

    vector<Student> students{};
    string outMessage{}, line{};
    int i{ 1 };

    while (getline(in, line) && (i <= 2000))
    {
        if (line == "") continue;
```

```

string surName{};
string name{};
string fatherName{};

char gender{};
int age{};
int year{};

size_t pid{ }, id{ line.find(';') };

surName = readSegment(line, pid, id);

if (!(surName.length() <= 12) && (surName.length() > 0))
{
    formatError(i, "surname", outFileName);
    return 0;
}

name = readSegment(line, pid, id);

if (!(name.length() <= 12) && (name.length() > 0))
{
    formatError(i, "name", outFileName);
    return 0;
}

fatherName = readSegment(line, pid, id);

if (!(fatherName.length() <= 12) && (fatherName.length() > 0))
{
    formatError(i, "father name", outFileName);
    return 0;
}

set<char> genderAllowed{ 'M', 'F' };
string sgender{ readSegment(line, pid, id) };

if (sgender.length() == 1)
{
    if (genderAllowed.contains(sgender[0]))
    {
        gender = sgender[0];
    }
    else
    {
        formatError(i, "gender", outFileName);
        return 0;
    }
}
else
{
    formatError(i, "gender", outFileName);
    return 0;
}

string sage{ readSegment(line, pid, id) };

if ((sage.length() == 2) && (isNumeric(sage)))
{
    age = stoi(sage);

    if (!(age >= 16) && (age <= 35))
    {
        formatError(i, "age", outFileName);
        return 0;
    }
}

```

```

else
{
    formatError(i, "age", outFileName);
    return 0;
}

string syear{ readSegment(line, pid, id) };
if ((syear.length() == 1) && (isNumeric(syear)))
{
    year = stoi(syear);

    if (!(year >= 1) && (year <= 6))
    {
        formatError(i, "year", outFileName);
        return 0;
    }
}
else
{
    formatError(i, "year", outFileName);
    return 0;
}

Student student{ surName, name, fatherName, gender, age, year };
students.push_back(student);

i++;
}

// Results
if (students.size() > 0)
{
    // #1

    float maxp{ 0 };
    int maxpy{ 1 };

    for (int i{ 1 }; i <= 6; i++)
    {
        float c{}, mc{};

        for (Student s : students)
        {
            if (s.year == i)
            {
                if (s.gender == 'M')    mc++;
                c++;
            }
        }

        float p{ mc / c };

        if (maxp < p)
        {
            maxp = p;
            maxpy = i;
        }
    }

    outMessage += string("Max men: ");

    if (maxp != 0)    outMessage += to_string(maxpy) + string(" year\n");
    else              outMessage += "no men in list\n";

    // #2

```

```

set<string> names{};
set<string> mostPopularNames{};

for (Student s : students)
{
    if (s.gender == 'M')        names.insert(s.name);
}

for (int i{}; i < 3; i++)
{
    int maxc{ -1 };
    string maxcn{};

    for (string name : names)
    {
        if (!mostPopularNames.contains(name))
        {
            int c{};

            for (Student s : students)
            {
                if (s.name == name) c++;
            }

            if (c > maxc)
            {
                maxc = c;
                maxcn = name;
            }
        }
    }

    mostPopularNames.insert(maxcn);
}

outMessage += string("Most popular male names: ");

if (!mostPopularNames.empty())
{
    for (string n : mostPopularNames) outMessage += n + " ";
}
else    outMessage += "no male names";

outMessage += "\n";

// #3

int mmina{ 100 }, fmina{ 100 };
string mminan{"no males"}, fminan{"no females"};

for (Student s : students)
{
    if (s.gender == 'M')
    {
        if (s.age < mmina)
        {
            mmina = s.age;
            mminan = s.name + " " + s.surName;
        }
    }
    else
    {
        if (s.age < fmina)
        {
            fmina = s.age;
            fminan = s.name + " " + s.surName;
        }
    }
}

```

```

        }
    }

    outMessage += string("Youngest male: ") + mminan + string("\n");
    outMessage += string("Youngest female: ") + fminan + string("\n");
}
else
{
    outMessage += string("The input file is empty");
}

// Output

ofstream out;
out.open(outFileName + ".txt");

if (out)
{
    out << outMessage;
    cout << outMessage;
}
else cout << "Output file open error" << endl;
}

bool isNumeric(string s)
{
    set<char> numbers{ '0', '1', '2', '3', '4', '5', '6', '7', '8', '9' };

    for (char c : s)
    {
        if (!numbers.contains(c)) return false;
    }

    return true;
}

string readSegment(string line, size_t& pid, size_t& id)
{
    string res{};

    if (id != string::npos)
    {
        if (pid != 0) pid++;
        res = line.substr(pid, id - pid);
    }

    pid = id;
    id = line.find(';', pid + 1);

    return res;
}

void formatError(int lineid, string segment, string outFileName)
{
    string outMessage{ string("line ") + to_string(lineid) + string(": format error") + segment + " " };

    ofstream out;
    out.open(outFileName + ".txt");

    if (out)
    {
        out << outMessage;
        cout << outMessage;
    }
    else cout << "Output file open error" << endl;
}

```


Тесты

Пример входного файла – “example_in1.txt” (приложен к документации)

Для понимания ожидаемых результатов приведен пример корректного входного файла на 50 строк. В тестах используются модификации строки этого файла.

Метод тестирования: покрытие элементарных указаний (белый ящик)

Блоки элементарных указаний:

- 1 [32-38]: string outFileName; ... in.open("in1.txt").
- 2 [42-44]: string outMessage{ "Input file open error" }; ...
out.open(outFileName + ".txt").
- 3 [48-49]: out << outMessage; cout << outMessage.
- 4 [51]: cout << "Output file open error" << endl.
- 5 [53]: return 0.
- 6 [56-59]: vector<Student> students{}; ... int i{ 1 }.
- 7 [63]: continue.
- 8 [65-71]: Объявления локальных переменных (surName...year).
- 9 [73-75]: size_t pid{ }, id{ line.find(';') }; ... surName =
readSegment(line, pid, id).
- 10 [79-80]: formatError(i, "surname", outFileName); return 0.
- 11 [83]: name = readSegment(line, pid, id).
- 12 [87-88]: formatError(i, "name", outFileName); return 0.
- 13 [91]: fatherName = readSegment(line, pid, id).
- 14 [95-96]: formatError(i, "father name", outFileName); return 0.
- 15 [100-101]: set<char> genderAllowed{ 'M', 'F' }; ... string sgender{
readSegment(line, pid, id) }.
- 16 [107]: gender = sgender[0].

```

17 [111-112]: formatError(i, "gender", outFileName); (символ) ... return
0.
18 [117-118]: formatError(i, "gender", outFileName); (длина) ... return
0.
19 [121]: string sage{ readSegment(line, pid, id) }.
20 [125]: age = stoi(sage).
21 [129-130]: formatError(i, "age", outFileName); (диап.) ... return 0.
22 [135-136]: formatError(i, "age", outFileName); (форм.) ... return 0.
23 [139]: string syear{ readSegment(line, pid, id)}.
24 [143]: year = stoi(syear).
25 [147-148]: formatError(i, "year", outFileName); (диап.) ... return 0.
26 [153-154]: formatError(i, "year", outFileName); (форм.) ... return 0.
27 [158-161]: Student student{ ... }; ... i++.
28 [170-171]: float maxp{ -1 }; int maxpy{ 1 }.
29 [175]: float c{}, mc{}.
30 [181]: mc++.
31 [182]: c++.
32 [186]: float p{ mc / c }.
33 [190-191]: maxp = p; maxpy = i.
34 [195]: outMessage += string("Max men: ").
35 [197]: outMessage += to_string(maxpy)... .
36 [198]: outMessage += "no men in list\n".
37 [202-203]: set<string> names{}...mostPopularNames{}.
38 [207]: names.insert(s.name).
39 [212-213]: int maxc{ -1 }; string maxcn{}.
40 [219]: int c{}.
41 [223]: c++.
42 [228-229]: maxc = c; maxcn = name.
43 [234]: mostPopularNames.insert(maxcn).
44 [237]: outMessage += string("Most popular male names: ").

```

```

45  [241]: outMessage += n + " ".
46  [243]: outMessage += "no male names".
47  [245]: outMessage += "\n".
48  [249-250]: int mmina{ 100 }...fminan{ "no females" }.
49  [258-259]: mmina = s.age; mminan = s.name + " " + s.surName.
50  [266-267]: fmina = s.age; fminan = s.name + " " + s.surName.
51  [272-273]: outMessage += ... (Youngest M); outMessage += ... (F).
52  [277]: outMessage += "The input file is empty".
53  [282-283]: ofstream out; out.open(outFileName + ".txt").
54  [287-288]: out << outMessage; cout << outMessage.
55  [290]: cout << "Output file open error" << endl.
56  [296]: set<char> numbers{ ... }.
57  [300]: return false.
58  [303]: return true.
59  [308]: string res{}.
60  [312]: if (pid != 0) pid++.
61  [313]: res = line.substr(...).
62  [316-319]: pid = id; ... return res.
63  [324-327]: string outMessage{ ... }; ... out.open(...).
64  [331-332]: out << outMessage; cout << outMessage.
65  [334]: cout << "Output file open error".

```

Тесты

- T1 Файл in1.txt переименован.
- T2 Строка 1: ;Ivan;Ivanovich;M;20;3; (Пустая фамилия).
- T3 Строка 1: Ivanov;;Ivanovich;M;20;3; (Пустое имя).
- T4 Строка 1: Ivanov;Ivan;;M;20;3 (Пустое отчество).
- T5 Строка 1: Ivanov;Ivan;Ivanovich;Male;20;3; (Длина поля пола > 1).
- T6 Строка 1: Ivanov;Ivan;Ivanovich;X;20;3; (Недопустимый символ пола).

- T7 Строка 1: Ivanov;Ivan;Ivanovich;M;5;3; (Возраст 1 цифра) или M;abc;3;.
- T8 Строка 1: Ivanov;Ivan;Ivanovich;M;15;3; (Возраст < 16).
- T9 Строка 1: Ivanov;Ivan;Ivanovich;M;20;10; (Курс 2 цифры).
- T10 Входной файл без изменений (корректные данные).
- T11 Входной файл пуст.
- T12 Входной файл без изменений. Невозможно записать в выходной файл.
- T13 Строка 1: Ivanov;Ivan;Ivanovich;M;18;0; (Курс вне диапазона 1-6).
- T14 Строка 1: ;Ivan;Ivanovich;M;20;3 (Пустая фамилия) Невозможно записать в выходной файл.
- T15 Строка 1: IvanovIvanIvanovichM203 (Отсутствуют разделители ;).
- T16 Файл in1.txt переименован. Невозможно записать в выходной файл.
- T17 Строка 1: [пусто].
- T18 В файле отсутствуют студенты, у которых gender = M.
- T19 В файле отсутствуют студенты, у которых gender = F.

«+» - блок элементарных указаний выполняется

«-» - блок элементарных указаний не выполняется

Таблица 2 – Таблица покрытия элементарных указаний

№ ЭУ	T1	T2	T3	T4	T5	T6	T7	T8	T9
1	+	+	+	+	+	+	+	+	+
2	+	-	-	-	-	-	-	-	-
3	+	-	-	-	-	-	-	-	-
4	-	-	-	-	-	-	-	-	-
5	+	-	-	-	-	-	-	-	-

6	-	+	+	+	+	+	+	+	+
7	-	-	-	-	-	-	-	-	-
8	-	+	+	+	+	+	+	+	+
9	-	+	+	+	+	+	+	+	+
10	-	+	-	-	-	-	-	-	-
11	-	-	+	+	+	+	+	+	+
12	-	-	+	-	-	-	-	-	-
13	-	-	-	+	+	+	+	+	+
14	-	-	-	+	-	-	-	-	-
15	-	-	-	-	+	+	+	+	+
16	-	-	-	-	-	-	-	-	-
17	-	-	-	-	-	+	-	-	-
18	-	-	-	-	+	-	-	-	-
19	-	-	-	-	-	-	+	+	+
20	-	-	-	-	-	-	-	+	+
21	-	-	-	-	-	-	-	+	-
22	-	-	-	-	-	-	+	-	-
23	-	-	-	-	-	-	-	-	+
24	-	-	-	-	-	-	-	-	-
25	-	-	-	-	-	-	-	-	-
26	-	-	-	-	-	-	-	-	+
27	-	-	-	-	-	-	-	-	-
28	-	-	-	-	-	-	-	-	-
29	-	-	-	-	-	-	-	-	-
30	-	-	-	-	-	-	-	-	-
31	-	-	-	-	-	-	-	-	-
32	-	-	-	-	-	-	-	-	-

33	-	-	-	-	-	-	-	-	-
34	-	-	-	-	-	-	-	-	-
35	-	-	-	-	-	-	-	-	-
36	-	-	-	-	-	-	-	-	-
37	-	-	-	-	-	-	-	-	-
38	-	-	-	-	-	-	-	-	-
39	-	-	-	-	-	-	-	-	-
40	-	-	-	-	-	-	-	-	-
41	-	-	-	-	-	-	-	-	-
42	-	-	-	-	-	-	-	-	-
43	-	-	-	-	-	-	-	-	-
44	-	-	-	-	-	-	-	-	-
45	-	-	-	-	-	-	-	-	-
46	-	-	-	-	-	-	-	-	-
47	-	-	-	-	-	-	-	-	-
48	-	-	-	-	-	-	-	-	-
49	-	-	-	-	-	-	-	-	-
50	-	-	-	-	-	-	-	-	-
51	-	-	-	-	-	-	-	-	-
52	-	-	-	-	-	-	-	-	-
53	-	-	-	-	-	-	-	-	-
54	-	-	-	-	-	-	-	-	-
55	-	-	-	-	-	-	-	-	-
isNumeric									
56	-	-	-	-	-	-	+	-	-
57	-	-	-	-	-	-	+	-	-
58	-	-	-	-	-	-	-	-	-

readSegment									
59	-	+	+	+	+	+	+	+	+
60	-	-	+	+	+	+	+	+	+
61	-	+	+	+	+	+	+	+	+
62	-	+	+	+	+	+	+	+	+
formatError									
63	-	+	+	+	+	+	+	+	+
64	-	+	+	+	+	+	+	+	+
65	-	-	-	-	-	-	-	-	-

№ ЭУ	T10	T11	T12	T13	T14	T15	T16	T17	T18	T19
1	+	+	+	+	+	+	+	+	+	+
2	-	-	-	-	+	-	+	-	-	-
3	-	-	-	-	+	-	+	-	-	-
4	-	-	+	-	-	-	+	-	-	-
5	-	-	-	-	+	-	+	-	-	-
6	+	+	-	+	-	+	-	+	+	+
7	-	-	-	-	-	-	-	+	-	-
8	+	-	-	+	-	+	-	+	+	+
9	+	-	-	+	-	+	-	+	+	+
10	-	-	-	-	+	+	-	-	-	-
11	+	-	-	+	-	-	-	+	+	+
12	-	-	-	-	-	-	-	-	-	-
13	+	-	-	+	-	-	-	+	+	+
14	-	-	-	-	-	-	-	-	-	-
15	+	-	-	+	-	-	-	+	+	+

16	+	-	-	-	-	-	-	+	-	-
17	-	-	-	-	-	-	-	-	-	-
18	-	-	-	-	-	-	-	-	-	-
19	+	-	-	+	-	-	-	+	+	+
20	+	-	-	+	-	-	-	+	+	+
21	-	-	-	-	-	-	-	-	-	-
22	-	-	-	-	-	-	-	-	-	-
23	+	-	-	+	-	-	-	+	+	+
24	+	-	-	+	-	-	-	+	+	+
25	-	-	-	+	-	-	-	-	-	-
26	-	-	-	-	-	-	-	-	-	-
27	+	-	-	-	-	-	-	+	+	+
28	+	-	-	-	-	-	-	+	+	+
29	+	-	-	-	-	-	-	+	+	+
30	+	-	-	-	-	-	-	+	-	+
31	+	-	-	-	-	-	-	+	+	+
32	+	-	-	-	-	-	-	+	+	+
33	+	-	-	-	-	-	-	+	+	+
34	+	-	-	-	-	-	-	+	+	+
35	+	-	-	-	-	-	-	+	+	+
36	-	-	-	-	-	-	-	-	+	-
37	+	-	-	-	-	-	-	+	+	+
38	+	-	-	-	-	-	-	+	-	+
39	+	-	-	-	-	-	-	+	+	+
40	+	-	-	-	-	-	-	+	+	+
41	+	-	-	-	-	-	-	+	-	+
42	+	-	-	-	-	-	-	+	-	+

43	+	-	-	-	-	-	-	+	-	+
44	+	-	-	-	-	-	-	+	+	+
45	+	-	-	-	-	-	-	+	-	+
46	-	-	-	-	-	-	-	-	+	-
47	+	-	-	-	-	-	-	+	+	+
48	+	-	-	-	-	-	-	+	+	+
49	+	-	-	-	-	-	-	+	-	+
50	+	-	-	-	-	-	-	+	+	-
51	+	-	-	-	-	-	-	+	+	+
52	-	+	-	-	-	-	-	-	-	-
53	+	+	+	+	+	+	+	+	+	+
54	+	-	-	+	-	-	-	+	+	+
55	-	-	+	-	+	+	+	-	-	-
isNumeric										
56	+	-	-	+	-	-	-	+	+	+
57	-	-	-	-	-	-	-	-	-	-
58	+	-	-	+	-	-	-	+	+	+
readSegment										
59	+	-	-	+	-	+	-	+	+	+
60	+	-	-	+	-	-	-	+	+	+
61	+	-	-	+	-	+	-	+	+	+
62	+	-	-	+	-	+	-	+	+	+
formatError										
63	-	-	-	+	+	+	-	-	-	-
64	-	-	-	+	-	+	-	-	-	-
65	-	-	-	-	+	-	-	-	-	-

Таблица 3 – Тестовое множество

№	Input (Клавиатура)	Input (Файл in1.txt)	Output (Экран)	Output (Файл)
1	out	[файл переименован]	Input file open error	Input file open error
2	out	... ;Ivan;Ivanovich;M; 20;3; ...	line 1: format error (surname)	line 1: format error (surname)
3	out	... Ivanov;;Ivanovich; M;20;3; ...	line 1: format error (name)	line 1: format error (name)
4	out	... Ivanov;Ivan;;M;20; 3 ...	line 1: format error (father name)	line 1: format error (father name)
5	out	... Ivanov;Ivan;Ivanov ich;Male;20;3; ...	line 1: format error (gender)	line 1: format error (gender)
6	out	... Ivanov;Ivan;Ivanov ich;X;20;3; ...	line 1: format error (gender)	line 1: format error (gender)
7	out	... Ivanov;Ivan;Ivanov ich;M;5;3; ...	line 1: format error (age)	line 1: format error (age)
8	out	...	line 1: format	line 1: format

		Ivanov;Ivan;Ivanovich;M;15;3; ...	error (age)	error (age)
9	out	... Ivanov;Ivan;Ivanovich;M;20;10; ...	line 1: format error (year)	line 1: format error (year)
10	out	[Корректный файл]	Max men: 5 year Most popular male names: Ale Pav Vas Youngest male: Ivan Ivanov Youngest female: Petr Petrov	Max men: 5 year Most popular male names: Ale Pav Vas Youngest male: Ivan Ivanov Youngest female: Petr Petrov
11	out	[Пустой файл]	The input file is empty	The input file is empty
12	out	[Корректный файл]	Output file open error	[Файл не создан]
13	out	... Ivanov;Ivan;Ivanovich;M;18;0; ...	line 1: format error (year)	line 1: format error (year)
14	out	... ;Ivan;Ivanovich;M; 20;3; ...	Output file open error	[Файл не создан]
15	out	... IvanovIvanIvanovich	line 1: format error (surname)	line 1: format error (surname)

		hM203 ...		
16	out	[файл переименован]	Output file open error	[Файл не создан]
17	out	... [Пустая строка] ...	Max men: 5 year Most popular male names: Ale Pav Vas Youngest male: Koz Kozlov Youngest female: Petr Petrov	Max men: 5 year Most popular male names: Ale Pav Vas Youngest male: Koz Kozlov Youngest female: Petr Petrov
18	out	[У всех студентов gender = M]	Max men: 1 year Most popular male names: Ale Pav Vas Youngest male: Petr Petrov Youngest female: no females	Max men: 1 year Most popular male names: Ale Pav Vas Youngest male: Petr Petrov Youngest female: no females
19	out	[У всех студентов gender = F]	Max men: no men in list Most popular male names: no male names Youngest male: no males Youngest female: Petr Petrov	Max men: no men in list Most popular male names: no male names Youngest male: no males Youngest female: Petr Petrov

Тестирование программы

Таблица 4 – Результаты тестирования

№	Результат (Экран)	Результат (Файл `out.txt`)
1	Input file open error	Input file open error
2	line 1: format error (surname)	line 1: format error (surname)
3	line 1: format error (name)	line 1: format error (name)
4	line 1: format error (father name)	line 1: format error (father name)
5	line 1: format error (gender)	line 1: format error (gender)
6	line 1: format error (gender)	line 1: format error (gender)
7	line 1: format error (age)	line 1: format error (age)
8	line 1: format error (age)	line 1: format error (age)
9	line 1: format error (year)	line 1: format error (year)
10	Max men: 5 year Most popular male names: Ale Pav Vas Youngest male: Ivan Ivanov Youngest female: Petr Petrov	Max men: 5 year Most popular male names: Ale Pav Vas Youngest male: Ivan Ivanov Youngest female: Petr Petrov
11	The input file is empty	The input file is empty
12	Output file open error	[Файл не создан]
13	line 1: format error (year)	line 1: format error (year)
14	Output file open error	[Файл не создан]
15	line 1: format error (surname)	line 1: format error (surname)
16	Output file open error	[Файл не создан]
17	Max men: 5 year Most popular male names: Ale Pav Vas Youngest male: Koz Kozlov Youngest female: Petr Petrov	Max men: 5 year Most popular male names: Ale Pav Vas Youngest male: Koz Kozlov Youngest female: Petr Petrov

18	Max men: 1 year Most popular male names: Ale Pav Vas Youngest male: Petr Petrov Youngest female: no females	Max men: 1 year Most popular male names: Ale Pav Vas Youngest male: Petr Petrov Youngest female: no females
19	Max men: no men in list Most popular male names: no male names Youngest male: no males Youngest female: Petr Petrov	Max men: no men in list Most popular male names: no male names Youngest male: no males Youngest female: Petr Petrov

Список использованных источников

- 1 Страуструп Б. Язык программирования C++. 4-е изд. – М.: Бином. Лаборатория знаний, 2022.